

基于本地知识库的论文搜索助手系统 — 算法方案文档

课程: 数据库系统实验 完成日期: 2026年6月

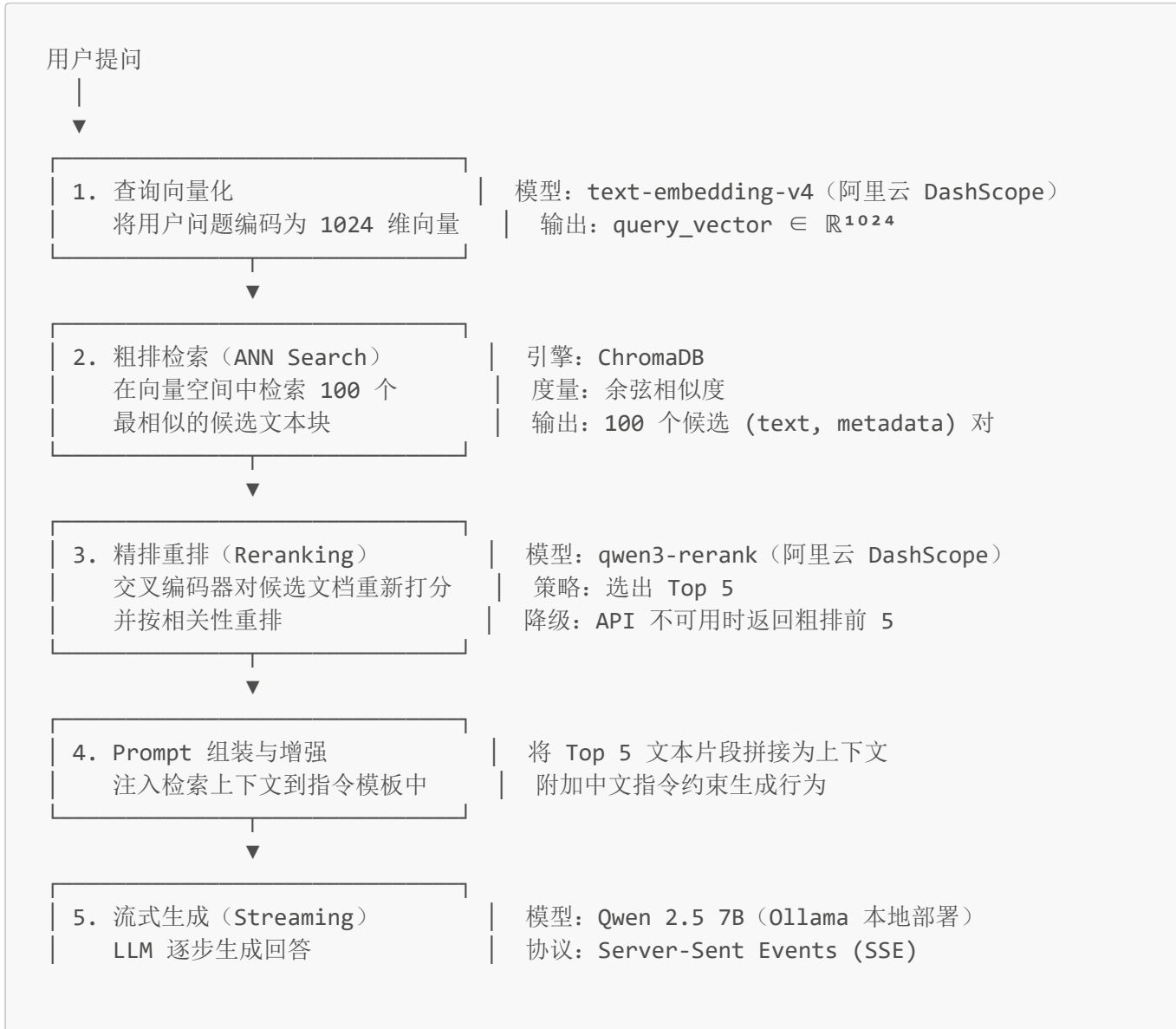
一、引言

本系统的核心智能能力建立在检索增强生成 (Retrieval-Augmented Generation, RAG) 技术框架之上。RAG 是一种将信息检索与大语言模型生成相结合的范式，通过在生成阶段向模型注入从外部知识库中检索到的相关文本，使模型的输出更加准确、有据可查，并有效缓解大语言模型的“幻觉”问题。

本系统的算法方案涵盖以下技术环节：文档解析与结构化提取、文本分块策略、向量化嵌入、两阶段检索（粗排 + 精排）、检索增强 Prompt 工程、流式生成与传输、标题自动提取，以及检索质量的自动化评估方法。以下逐一展开说明。

二、RAG 管线总体架构

系统的 RAG 管线可以抽象为一条从“用户提问”到“AI 回答”的处理流水线，各阶段的数据流转如下：



前置来源标记后流式输出	格式：逐 token 输出
-------------	---------------

以下各节分别对管线中的每个环节以及相关的辅助技术方案进行详细阐述。

三、文档解析方案

3.1 双格式支持

系统通过 `utils/parser.py` 模块支持 PDF 和 DOCX 两种学术论文格式的解析，分别采用 `pdfplumber` 和 `python-docx` 两个库作为底层解析引擎。

PDF 解析 (`parse_pdf` 函数)：逐页遍历 PDF 文档，对每一页分别提取纯文本和表格数据。文本提取时在每页内容前加入 `--- Page N ---` 标记，便于后续定位来源页码。表格提取以二维矩阵的形式返回（行 × 列的字符串列表），保留原始表格的结构信息。

DOCX 解析 (`parse_docx` 函数)：遍历文档中的所有段落元素，过滤空段落，收集非空段落文本。同时提取文档中的所有表格，同样以二维矩阵形式返回。

3.2 表格的 Markdown 化

论文中的表格包含大量结构化信息（实验结果、对比数据等），对问答质量至关重要。系统将提取到的表格矩阵通过 `table_to_markdown` 函数转换为 Markdown 格式文本。转换过程处理了若干边界情况：单元格内的换行符被替换为空格（防止 Markdown 表格结构被破坏）、短行被填充至与表头等宽、长行被截断至表头列数。转换后的 Markdown 表格作为一个独立的知识块存入向量数据库，保留了原始的结构化语义。

3.3 论文标题自动提取

系统实现了多策略级联的标题提取方案 (`extract_title` 函数)，从文档中自动识别论文标题，以在论文库列表中提供更有意义的显示名称。

PDF 标题提取策略：优先读取 PDF 文档元数据中的 `/Title` 字段。大多数正式发表的英文学术论文都在元数据中嵌入了准确的标题信息，此策略命中率高且结果可靠。若元数据为空或不含标题，则降级到基于首页文本的启发式提取。

DOCX 标题提取策略：优先读取 `core_properties.title` 属性。若不存在，则降级到基于文档开头段落的启发式提取。

启发式文本提取 (`_extract_title_from_text` 函数)：当元数据策略失败时，系统分析文档首页（或 DOCX 开头段落）的前 8 行文本，通过噪声过滤器 (`_is_noise_line` 函数) 剔除页码、卷期号、DOI 编号、URL、"Abstract"、"Keywords" 等常见非标题行，然后从剩余行中选取最长的有意义行作为标题。如果第二行的长度达到第一行的 40% 以上（表明标题可能被分行排版），则将前两行合并作为标题。标题长度上限设为 200 个字符。

噪声过滤器 (`_is_noise_line` 函数)：基于正则表达式匹配，过滤掉以下类型的行——长度小于 5 个字符的短行、纯数字行、包含 "IEEE"、"ACM"、"Springer"、"DOI"、"Vol."、"No." 等学术会议/出版商标识的行、包含 URL 的行、以及 "Abstract"、"Keywords"、"Introduction" 等论文结构性标记行。

兜底策略：如果以上所有策略均未能提取到有效标题，则使用去掉文件扩展名后的文件名作为标题。

四、文本分块方案

4.1 分块策略

大语言模型和嵌入模型对输入长度有上限约束，同时过长的文本块会稀释向量表示的语义密度。系统将解析后的论文文本切分为较小的知识块（chunk），采用 LangChain 提供的 `RecursiveCharacterTextSplitter` 进行递归分块。

核心参数设置如下：

参数	取值	说明
chunk_size	400 字符	每个知识块的目标长度
chunk_overlap	50 字符	相邻块之间的重叠长度
separators	<code>["\n\n", "\n", "。", "!", "?", ";", ",", " ", " "]</code>	中文友好的分隔符序列

分隔符按照从粗到细的优先级排列：优先在段落分隔处切分（`\n\n`），其次在换行处切分（`\n`），再依次尝试在句号、感叹号、问号、分号、逗号处切分。这种设计使得分块结果尽量保持语义完整性，避免在句子中间被截断。50 个字符的重叠保证了跨块边界的上下文不会完全丢失。

4.2 超长块的安全阀机制

阿里云 DashScope 的 `text-embedding-v4` 模型对输入文本有 8192 token 的硬限制。虽然 400 字符的目标分块大小通常远低于此限制，但表格转换后的 Markdown 文本可能生成较长的单一知识块。系统为此增加了安全阀逻辑：对任何超过 7500 字符的知识块执行强制二次分割，确保所有输入嵌入模型的文本块都不会超过 token 上限。

4.3 表格块的特殊处理

与段落文本的分块策略不同，表格经过 Markdown 化处理后作为一个独立的知识块直接入库，不参与递归分块。这一设计保留了表格的完整性——如果被拆分，表格的行列结构将失去语义。

五、向量化嵌入方案

5.1 嵌入模型选择

系统选用阿里云 DashScope 平台提供的 `text-embedding-v4` 模型进行文本向量化。该模型输出 1024 维的稠密向量，在中文语义理解任务上表现优秀，且支持通过 OpenAI 兼容 SDK 调用，降低了集成成本。

5.2 批量嵌入策略

嵌入服务（`embed.py` 中的 `get_text_embeddings` 函数）将输入文本列表按每批 10 条进行切分，逐批调用 DashScope API。选择 10 作为批次大小是因为 DashScope 对单次嵌入请求的文本数量上限为 10 条。函数将所有批次的结果拼接为完整的向量列表返回。

5.3 入库与查询的统一嵌入

无论是文档入库时的文本块向量化，还是用户查询时的查询向量化，都使用相同的 `text-embedding-v4` 模型。这保证了查询向量和文档向量处于同一语义空间中，余弦相似度计算有意义。

六、两阶段检索方案

这是系统 RAG 管线中最关键的算法环节。系统采用“粗排-精排”两阶段检索策略，在保证检索覆盖面的同时提升最终输入 LLM 的上下文质量。

6.1 第一阶段：粗排检索（Coarse Retrieval）

粗排阶段使用 ChromaDB 向量数据库执行近似最近邻（Approximate Nearest Neighbor, ANN）搜索。具体流程如下：

1. 将用户查询文本通过 `text-embedding-v4` 模型编码为 1024 维查询向量。
2. 在 ChromaDB 的 `my_knowledge_base` 集合上执行 `.query(n_results=100)` 操作，返回与查询向量余弦距离最近的 100 个候选文本块及其元数据（`source` 字段标识来源论文文件名）。

选择 100 作为粗排候选数量是一个经验性的平衡：足够大以覆盖可能的语义相关片段（特别是在知识库包含多篇论文、数千个文本块的情况下），又不会给后续精排阶段带来过大的计算负担。

6.2 第二阶段：精排重排（Fine Reranking）

粗排基于向量空间的余弦距离进行快速召回，但这种双编码器（Bi-Encoder）模式的表示能力有限——查询和文档被独立编码后再计算相似度，无法捕捉查询与文档之间的深层交互语义。精排阶段引入交叉编码器（Cross-Encoder）模型进行重排序，显著提升最终检索质量。

系统使用阿里云 DashScope 提供的 `qwen3-rerank` 交叉编码器模型。与双编码器不同，交叉编码器将查询和候选文档拼接为一个输入序列，通过注意力机制直接建模两者之间的语义关联，输出一个标量相关性分数。这种方式虽然计算成本更高（需要对每对查询-文档组合进行一次前向推理），但排序精度显著优于双编码器。

精排的具体流程如下：

1. 将用户查询、100 个粗排候选文本块及其元数据组装为请求体，发送到 DashScope 的 `/reranks` 接口。
2. API 返回一个结果列表，每个结果包含一个 `index` 字段（指向原始候选列表中的位置）和相关性分数。
3. 系统根据 `index` 对文档和元数据进行重排序，取前 `top_n` 个结果（默认 `top_n=5`）。

优雅降级机制：如果 Qwen3-Rerank API 不可用（网络超时、服务降级等），精排函数会捕获异常并直接返回粗排结果的前 `top_n` 个候选，确保整个 RAG 管线不会因为单一外部服务的故障而中断。

6.3 检索参数调优

系统在 v1.2 版本中将精排参数 `final_top_k` 从 2 调整为 5，调优依据如下：每个知识块约 400 字符，5 个块合计约 2000 字符（约 800-1000 token），对 Qwen 2.5 7B 模型的上下文窗口来说既提供了充分的参考信息，又不会引入过多噪声稀释模型的注意力。实验观察表明，Top 5 的设置回答质量和来源覆盖度上均优于 Top 2。

6.4 来源提取

精排完成后，系统从最终选出的文本块的元数据中提取 `source` 字段（即论文文件名），经过去重后得到引用来源列表。这些来源信息通过 `[[SOURCES:file1.pdf,file2.pdf]]` 标记嵌入到流式输出中，供前端解析和展示。

七、Prompt 工程方案

7.1 检索增强 Prompt 模板

当 RAG 检索阶段返回了相关文本片段时，系统将用户原始问题包装在一个精心设计的中文指令模板中。Prompt 模板的核心设计思想如下：

角色设定：指示 LLM 扮演“基于参考材料回答问题的助手”，明确其回答应建立在提供的参考资料之上。

参考材料注入：将 Top 5 检索结果以 `---相关片段---` 为分隔符拼接为一段完整的上下文文本，嵌入到 Prompt 的参考材料区域。

行为约束：明确指示 LLM 遵循以下规则——基于参考材料回答问题；如果参考材料中没有相关信息，应如实说明而非编造；保持回答的准确性和可靠性。

7.2 无检索结果时的降级处理

当 RAG 检索未返回任何相关文本（例如知识库为空或查询与知识库内容完全不相关）时，系统将用户的原始问题直接传递给 LLM，不注入任何参考材料。此时 LLM 将依赖自身参数知识进行回答，但由于缺少外部知识支撑，回答的可信度较低。

7.3 多轮对话上下文维护

系统将完整的对话历史（包含用户消息和 AI 回复的交替序列）连同增强后的 Prompt 一起发送给 LLM。这使得模型能够理解对话的上下文，支持追问、澄清和话题延续等多轮对话场景。对话历史以 OpenAI 兼容的消息列表格式传递，每条消息包含 `role`（"user" 或 "assistant"）和 `content` 两个字段。

八、流式生成与传输方案

8.1 LLM 流式生成

系统通过 Ollama 部署的 Qwen 2.5 7B 模型进行文本生成，调用其 OpenAI 兼容的 `/v1/chat/completions` 接口并设置 `stream=True` 参数。模型以逐 token 的方式返回生成结果，每个 token 作为一个 SSE（Server-Sent Events）数据块发送。

8.2 来源标记前置

在流式输出的起始位置，系统插入了一个自定义的来源标记字符串 `[[SOURCES:file1.pdf,file2.pdf]]`。这个标记先于 LLM 生成的任何文本到达前端，前端通过正则表达式提取后将其从显示内容中移除，并解析出引用的论文文件名用于展示。

8.3 SSE 事件协议

主后端作为中间层，将算法层的流式响应以 `text/event-stream` 格式转发给前端。SSE 协议定义了以下几种事件类型：

内容事件：携带 LLM 生成的文本片段（`delta.content`），前端将其追加到当前消息的末尾。

元事件（`type: "meta"`）：携带服务端自动创建的 `conversation_id`，前端据此更新当前会话标识。这是实现“首条消息自动创建会话”功能的关键机制。

结束标记 ([DONE]): 标记流式传输结束，前端据此恢复输入框的可用状态。

8.4 累积回复持久化

主后端在转发流式响应的同时，在内存中累积助手回复的完整内容。流结束后，使用独立的数据库会话将完整的回复文本保存到 MySQL 的 `messages` 表中。使用独立会话是为了避免 SSE 生成器的长生命周期与原始请求的数据库会话生命周期冲突。

九、检索质量评估方案

系统提供了一个自动化评估脚本 (`my_evaluate.py`)，用于定量衡量 RAG 管线的检索质量和生成质量。评估基于 50 篇论文的知识库和 `questions.json` 中预定义的 50 个评测问题。

9.1 检索阶段评估指标

评估检索阶段能力的三个指标：

Precision@K (精确率@K)：在系统检索返回的 Top K 个文本块中，有多少比例与标准答案 (ground truth) 相关。计算公式为 $\text{Precision@K} = |\text{检索结果} \cap \text{相关文档}| / K$ 。该指标衡量检索结果的纯净度——返回的结果中有多少是有用的。

Recall@K (召回率@K)：在所有相关文档中，有多少比例被系统检索到了 Top K 结果中。计算公式为 $\text{Recall@K} = |\text{检索结果} \cap \text{相关文档}| / |\text{相关文档总数}|$ 。该指标衡量检索的覆盖面——有多少应该被找到的内容确实被找到了。

F1@K：Precision@K 和 Recall@K 的调和平均值，综合反映检索质量。计算公式为 $\text{F1@K} = 2 \times \text{Precision@K} \times \text{Recall@K} / (\text{Precision@K} + \text{Recall@K})$ 。

9.2 生成阶段评估指标

评估生成阶段能力的三个指标：

BLEU (Bilingual Evaluation Understudy)：通过计算生成文本与参考文本之间的 n-gram 重叠度来评估文本相似性。BLEU 关注的是精确率维度——生成文本中有多少词出现在参考文本中。该指标在机器翻译领域广泛使用，但在问答场景中对语义相似性的捕捉能力有限。

ROUGE-L (Recall-Oriented Understudy for Gisting Evaluation - Longest)：基于最长公共子序列 (LCS) 衡量生成文本与参考文本之间的重叠程度。ROUGE-L 关注的是召回率维度——参考文本中有多少内容被生成文本覆盖了。相比 BLEU，ROUGE-L 对词序的要求更灵活。

BERTScore：使用预训练语言模型 (如 BERT) 的上下文嵌入来计算生成文本和参考文本之间的语义相似度。与 BLEU 和 ROUGE 依赖表面词匹配不同，BERTScore 在语义层面进行对齐，即使生成文本使用了不同的词汇，只要语义相近就能获得较高分数。BERTScore 的 F1 值综合了精确率和召回率两个维度。

9.3 本地评测结果

在 50 篇论文知识库上的评测结果如下：

指标	数值	说明
Precision@5	0.1480	返回的 5 个结果中平均约 0.74 个与标准答案直接相关

指标	数值	说明
Recall@5	0.7400	标准答案中的相关信息有 74% 被 Top 5 结果覆盖
F1@5	0.2467	综合检索质量分数
BLEU	0.0004	表面词匹配度较低（问答场景正常现象）
ROUGE-L	0.0011	表面序列重叠度较低
BERTScore F1	0.4532	语义相似度接近 0.45，表明生成内容在语义层面与参考答案有一定对齐

对结果的分析：Recall@5 达到 0.74 表明两阶段检索策略在召回能力上表现良好——大部分相关信息都能被检索到。Precision@5 相对较低是因为知识库中 50 篇论文的文本块数量庞大，Top 5 结果中不可避免地包含一些噪声。BLEU 和 ROUGE-L 的绝对值较低是问答场景下的正常现象——AI 生成的回答通常不会与标准答案逐字匹配，而是以不同的表述传达相似的语义。BERTScore 达到 0.45 证实了这一点，表明在语义嵌入空间中生成文本与参考答案存在有意义的相似性。

十、技术方案总结

本系统的算法方案可以用以下关键词概括：

RAG（检索增强生成） 是系统的核心范式，通过将外部知识库的检索结果注入 LLM 的生成过程，使回答建立在真实论文内容之上，降低了幻觉风险。

两阶段检索（粗排 + 精排） 是检索质量的保障——双编码器的 ANN 粗排保证了检索覆盖面和速度，交叉编码器的精排提升了最终上下文的相关性。

中文友好的文本分块 通过 400 字符的目标块大小、中文标点优先的分隔符序列和 50 字符的重叠，在语义完整性和块粒度之间取得了平衡。

多策略级联的标题提取 从元数据到首页文本到文件名兜底，覆盖了不同格式和质量论文的场景。

流式传输与持久化 通过 SSE 协议实现了低延迟的逐字输出体验，同时在流结束后保证了消息的完整持久化。

自动化评估体系 从检索精确率/召回率和生成语义相似度两个维度对系统进行了定量评估，为后续的参数调优和方案改进提供了数据基础。

后续可进一步探索的方向包括：引入 BM25 关键词检索与向量检索的混合策略、使用 HyDE（Hypothetical Document Embedding）技术提升查询质量、以及采用更精细的评测框架（如人工评分）来补充自动化指标的不足。